G - Global.

Syntax:

```
G/pattern/commands...
G~/pattern/commands...
```

Examples:

```
g/hello/p
g/color/s//colour/p
g/\C.$/ +1zu1/./ p
g~/^KEEP/d
```

Where:

`pattern`
: is any FRED pattern.

`commands`
: is a list of zero or more FRED commands. FRED will execute this list of commands for every line that contains a match for the given pattern. The end of this command list is indicated by a new-line.

`~`
: tells FRED that the command list should be executed for every line that does *not* contain a match for the given pattern.

Description:

`G` executes a set of commands on a set of lines that match or do not match a given pattern. If you do not supply an address, FRED will look at all the lines in the buffer to see if they match the pattern. If you specify one address, FRED will only look at that line. If you specify two addresses, FRED will look at all lines between and including the given addresses. In this case, the first address should precede the second.

`G` begins by copying its list of commands into a hidden buffer. Stream directives such as `\B`, `\L`, `\N`, etc. are expanded in the process. FRED copies the new-line that ends the list of commands into the hidden buffer along with everything else. Any valid command may be included in the command list.

Next FRED checks every line in the appropriate range and invisibly "marks" the line if it contains a match for the given pattern. FRED then executes the commands in the hidden buffer for each marked line. This means that FRED sets `"."` to the next marked line in the current buffer and executes the commands in the hidden buffer.

If the commands being executed for one marked line modify a second marked line, the internal "mark" on the second line is removed and `G` will not execute the hidden buffer on that second line. Commands that remove the "mark" from a line include

```
C D G M S ZB ZI
ZL ZM ZO ZS ZT ZU
```

As an example,

```
1,$-1g/^/ s/^/X/ .+1s/^/Y/
```

operates in the following manner.

1. `G` saves the two `S` commands in the hidden buffer, then goes through the current buffer marking every line, since every line matches the null beginning character `^`.

2. The hidden buffer is invoked for the first marked line, line 1. The first `S` command puts an `X` at the beginning of line 1, and the second `S` command puts a `Y` at the beginning of line 2. In the process, the "mark" on line two is removed since the line has been modified.
3. When `G` looks for the next marked line, it will find line 3. Continuing in this way, `G` will place an `X` at the start of each odd-numbered line and a `Y` at the start of each even-numbered line.

After each execution of the hidden buffer, FRED resets the null pattern `//` (the most recent pattern encountered) to the pattern in the `G` command. Thus

```
g/abc/ s//def/ s/ghi/jkl/
```

is equivalent to

```
g/abc/ s/abc/def/ s/ghi/jkl/
```

When `G` is finished, `"."` has the value it had after the last command executed from the hidden buffer. In addition, FRED sets the count register to the number of lines matched by the `G` command. Thus a command like

```
g/%/
```

sets the count register to the number of lines that contain a `%`. In this case, the closing `/` can be omitted if `O-S/` is in effect.

If you want the list of commands to contain a new-line character, put a `\C` in front of the new-line so that FRED doesn't think the new-line is the end of the command list. If you use a `\B` construction to put the contents of a buffer into the list of commands, FRED will not mistake new-line characters in the buffer for the new-line that ends the `G` command. FRED keeps track of the "input level" of new-line characters.

`G` executes its command list on marked lines in the order that the lines occur in the current buffer. The search for the next marked line begins wherever `"."` was when the hidden buffer finishes execution for the previous line, wrapping around from the bottom to the top of the buffer if necessary. This can make for quite a long search, especially `G`'s commands have moved `"."` beyond the location of the next marked line. You can increase the efficiency of a `G` command if you make sure that the hidden buffer does not move `"."` beyond the next marked line.